

```
ctkmu c -s7 -trsa -n verity_key0 -aEDSV -z2048 --rsaPubVal 10001
ctkmu c -s7 -trsa -n verity_key1 -aEDSV -z2048 --rsaPubVal 10001
ctkmu c -s7 -trsa -n verity_key2 -aEDSV -z2048 --rsaPubVal 10001
ctkmu c -s7 -trsa -n update_key -aEDSV -z2048 --rsaPubVal 10001
```

```
ctkmu x -s7 -y -w "wrap" -n "verity_key0"
python transBase64tohex.py
mv test_dec.pem verity_key0.en
openssl rsa -pubin -inform DER -RSAPublicKey_out -in verity_key0.der -out verity_key0.pem
```

```
ctkmu x -s7 -y -w "wrap" -n "verity_key1"
python transBase64tohex.py
mv test_dec.pem verity_key1.en
./encdecdata -s7 -w wrap -i verity_key1.en -o verity_key1.der
openssl rsa -pubin -inform DER -RSAPublicKey_out -in verity_key1.der -out verity_key1.pem
```

```
ctkmu x -s7 -y -w "wrap" -n "verity_key2"
python transBase64tohex.py
mv test_dec.pem verity_key2.en
./encdecdata -s7 -w wrap -i verity_key2.en -o verity_key2.der
openssl rsa -pubin -inform DER -RSAPublicKey_out -in verity_key2.der -out verity_key2.pem
```

```
ctkmu x -s7 -y -w "wrap" -n "update_key"
python transBase64tohex.py
mv test_dec.pem update_key.en
./encdecdata -s7 -w wrap -i update_key.en -o update_key.der
openssl rsa -pubin -inform DER -RSAPublicKey_out -in update_key.der -out update_key.pem
```

HOW TO VERIFY BY OPENSLL

key name:pubkey;
hashfileName:hash.bin
signFileName:sign.bin

hash:
openssl dgst -sha256 -hex -out hash.bin -binary data.txt

sign:
./signverdatakp -s7 -p verity_key0 -q verity_key0 -i hash.bin -o sign.bin

change key to PKCS8 format key:
1.openssl rsa -pubin -RSAPublicKey_in -in pubkey -out pubkeyPKCS8

verify

2.openssl rsautl -verify -inkey pubkeyPKCS8 -pubin -in sign.bin -out verifyResult.bin

Compare verify result

3.cmp hash.bin verifyResult.bin

Sign the plaintext using the private key:

openssl rsautl -sign -in info.txt -inkey privkey.pem -out info.txt.sign

Use the public key to verify the plaintext:

openssl rsautl -verify -in info.txt.sign -inkey pubkey.pem -pubin -out info_verify.txt